

DSA ASSIGNMENT -

Describe greedy algorithm with example.

Greedy Algorithm solves problems by making locally optimal choices to find the suitable global optimum.

Ex: Coin changing Problem.

* Given coins of denominations 1, 5, 10 and 25 find the min. no. of coins needed to make 37.

ALGORITHM:

1. Choose the largest coin (25) until it's no longer largest.
2. Then choose the next largest coin (10) until it's no longer possible.
3. Then choose the next largest coin (5), if next is largest then (5) change the next largest as that no.
4. Choose the smallest coin (1) until it's the remaining amount.

Result: $25 + 10 + 5 + 1 = 37$ (4 coins)

Greedy algorithm Time Complexity: $O(n)$

2. For all those algorithm, what is the time complexity, space complexity and Data Structure in use?

Ans: Greedy algorithm:

- i) Time Complexity: $O(n)$
- ii) Space Complexity: $O(1)$
- iii) Data Structure: Array/List.

Dijkstra's Algorithm:

- i) Time complexity - $O((E + V) \log V)$
- ii) Space complexity - $O(V)$
- iii) Data Structure - priority queue, graph.

Dynamic Program:

- i) Time complexity - $O(n)$
- ii) Space complexity - $O(n)$
- iii) Data Structure - Array.

Knap sack :

- i) Time complexity - $O(2^n)$
- ii) Space complexity - .
- iii) Data Structure - 2D Array.

3. What is Dynamic Program? What are the types of diff. algorithm?

Ans It is an algorithm technology for solving optimizing problem by breaking it down into simpler sub problems and leverage in the optimal solution to sub-problem to obtain the overall/global optimal solution.

Types:

- i) Top-Down Approach (Memorisation)
- ii) Bottom-Up Approach (tabulation)

Top-Down (Memorization): Solves the problem recursively and store the solution of sub-problems to avoid overlapping.

Bottom-up (tabulation): solves all sub-problems in tabulation, iterating over them to build-up the solution for original problem.

Algorithms:

1. Fibonacci series
2. Knapsack problem
3. Longest Common Sequence (LCS)
4. Shortest path (Dijkstra's)

4. Dynamic Programming with Time complexity, space complexity and Data Structure.

1. Fibonacci Series:

- i) Time complexity - $O(n)$
- ii) Space complexity - $O(1)$
- iii) Data Structure - Array (or) Variables.

2. Knapsack problem:

- i) Time complexity - $O(n^2)$
- ii) Space complexity - $O(2^n)$
- iii) Data structure - 2D Array.

3. Longest Common Subsequence:

- i) Time complexity - $O(m*n)$ where m and n are length of two seq.
- ii) Space complexity - $O(m*n)$
- iii) Data structure - 2D Array.

4. Dijkstra's Algorithm:

- i) Time complexity - $O((E+V)\log V)$
- ii) Space complexity - $O(V)$
- iii) Data structure - Graph, priority queue.